AI or Ally? Understanding Student Reliance on GitHub Copilot and Human Peers Through Eye Tracking

George Salib and Zohreh Sharafi

***Abstract*—As AI-powered tools like GitHub Copilot become increasingly integrated into development workflows, it's essential to understand how developers—particularly novices—interact with these systems compared to traditional peer collaboration. This paper presents a within-subjects eye-tracking study of novice developers completing programming tasks with assistance from either GitHub Copilot or a human peer.**

We evaluate participants' task performance, visual attention patterns, and perceived reliance on the assistant. Results show that participants achieved significantly higher correctness scores with Copilot (50%) than with peers (32%), but also spent more time and exhibited higher cognitive load. Participants relied more frequently on Copilot, spending more time fixating on the AI interface, even though they visited the peer chat pane more frequently than Copilot's. Moreover, subjective responses indicated that Copilot was widely credited with enhancing speed and concentration, whereas opinions on its impact on code quality were more mixed. Visual attention patterns further indicated assistant-specific strategies, with significant differences in how participants navigated code panes, project structures, and error traces depending on the support type.

These findings highlight important trade-offs between AI and human support in programming workflows and offer design implications for future intelligent development environments that better balance correctness, trust, and cognitive effort.

***Index Terms*—GitHub Copilot, AI-assisted programming, Peer programming, Eye-tracking, Trust and Reliance in AI**

I. Introduction

THE adoption of AI-powered tools is rapidly expanding in software engineering (SE), where they are becoming integral to everyday development workflows.

A recent survey reports that 92% of developers use AI tools both professionally and personally [1]. These tools provide context-aware code suggestions, automate routine tasks, and assist with activities such as generating boilerplate code or refactoring functions. As their capabilities and accessibility grow, AI assistants increasingly shape how developers approach programming tasks.

Prior research has begun to examine AI assistants as pair-programming partners, highlighting both potential productivity benefits and risks such as overreliance or reduced code understanding [2]. However, despite the rapid adoption of these tools, how developers interact with them—and the cognitive consequences of such interactions—remains insufficiently understood.

Manuscript created September, 2025; Z. Sharafi, G. Salib are with the Department of Computer and Software Engineering, Polytechnique Montréal, Canada. E-mail: {george.salib,zohreh.sharafi}@polymtl.ca

This question is particularly relevant in educational settings. Large language model (LLM) tools are now widely used by students and professionals alike. Surveys indicate that most college students regularly use conversational LLMs for academic and coding tasks [3], while over 90% of developers report using LLMs in professional work [4]

As students increasingly learn programming alongside AI assistants, early interaction patterns may influence long-term professional practices. Educational studies also explore how AI-assisted programming affects motivation, programming anxiety, collaborative learning, and performance compared to traditional peer programming [5].

Understanding how different forms of assistance shape attention, reasoning, and reliance therefore has implications for both software engineering education and tool design.

Human–AI interaction research suggests that the use of automated assistance is shaped by users' attitudes toward automation. Trust influences whether and how users rely on automated systems [6], and effective use requires calibration between trust and system capability [7]. When this calibration is misaligned, users may exhibit *misuse* through overreliance or *disuse* by rejecting a capable system. In this context, *trust* refers to a user's internal attitude toward an assistance source, whereas *reliance* denotes the behavioral manifestation of that trust during task execution [6]. Prior work in software engineering further shows that awareness of a code's origin—human or machine—can influence developers' attention, reliance, and problem-solving strategies during debugging and code review tasks [8]–[10].

Despite these insights, much of the existing research evaluates AI programming tools primarily through automated benchmarks or isolated coding tasks. Such evaluations often overlook the human interaction processes that occur during real development activities, particularly collaborative settings where programmers rely on external assistance. Thus, reported improvements in AI model performance do not necessarily translate to how developers interact with these systems in practice.

Motivated by this gap, we examine how AI-based and human assistance shape developers' performance, visual attention, and reliance during debugging tasks. We adopt a mixed-methods approach that combines quantitative measures (*e.g.,* eye-tracking and task performance) with qualitative insights from post-task questionnaires, comparing how developers interact with and rely on GitHub Copilot versus a human peer.

To contextualize adoption and reliance behaviors, we draw

on established models of technology acceptance and their AI-specific extensions, which highlight collaborative intention and perceived usefulness as key drivers of adoption [11]. In software engineering, recent evidence further suggests that the adoption of LLM assistants is strongly influenced by performance expectancy and habit [12]. However, habitual reliance on AI assistance may also introduce risks, as the ease of access to generated suggestions can encourage users to bypass verification steps and accept outputs without sufficient scrutiny.

These dynamics are particularly relevant in programming contexts where external assistance plays a collaborative role. In AI-assisted pair programming, the assistant may act either as an asset, providing rapid candidate solutions, or as a liability, introducing subtle errors that require careful validation [2]. Although prior work suggests that AI assistance can improve short-term motivation and reduce programming anxiety, its effects on cognitive effort and collaborative reasoning differ from those observed in traditional human peer programming [5]. In addition, much of the empirical work on AI-assisted programming relies primarily on subjective measures such as surveys or interviews [13]–[15]. These approaches are susceptible to recall and observer biases [13], [16], and provide limited visibility into the cognitive processes that occur during task execution [17].

To address these limitations, researchers increasingly employ objective techniques such as *eye tracking*, which capture developers' gaze behavior during task execution and provide measurable indicators of visual attention [17]–[21]. In particular, fixation-based metrics provide insights into *cognitive load*, defined as the mental effort required for task processing, where longer and more frequent fixations are commonly associated with an increased cognitive load [17], [22], [23]. We use the term *visual attention patterns* to describe how gaze is distributed across interface elements, reflecting developers' information-seeking and problem-solving strategies [19].

Building on these insights, we examine how developers integrate external assistance during programming tasks by combining behavioral measures of attention with performance outcomes. We operationalize reliance using the *attention ratio*, defined as the proportion of time spent attending to the assistant relative to the code, where higher values indicate greater behavioral dependence on external suggestions [6], [20].

We investigate whether different forms of assistance are associated with distinct patterns of cognitive load, reliance behavior, and visual navigation during debugging tasks. Understanding these differences can inform both programming education and the design of intelligent development environments that better support calibrated human–AI collaboration.

Based on prior work on trust, reliance, and gaze behavior in software tasks [6]–[8], [19], [24], we hypothesize that assistance type will produce distinct gaze behaviors. AI assistance is expected to increase attention to the suggestion pane, while human assistance may involve more frequent gaze switching between communication and code. We also expect AI-assisted work to induce greater verification effort, reflected in higher cognitive load.

This framing guides the following research questions:

RQ1. What effect does the assistance type have on participants' performance during task completion?

RQ2. How does the type of assistance (human peer vs. GitHub Copilot) influence participants' reliance on that assistance during task completion?

RQ3. Can we differentiate the assistance type (human peer vs. GitHub Copilot) based on their visual attention patterns?

We recruited 50 participants who completed two sets of programming tasks: one with assistance from a human peer and one with GitHub Copilot. We collected objective metrics, including performance and eye-tracking data, alongside subjective feedback from post-task questionnaires. By directly contrasting AI assistance with peer collaboration, we address the lack of collaborative comparisons in prior work and enable a more realistic assessment of performance and interaction dynamics in software engineering tasks [15], [25].

Our results show a trade-off between performance and effort: participants achieved higher accuracy with GitHub Copilot (50%) compared to a peer (32%), though this came at the cost of slightly longer task durations and greater cognitive load. Participants relied more heavily on Copilot, spending significantly more time fixating on and interacting with the AI interface even though their visits to the peer chat pane were shorter but more frequent. Also, participants favored Copilot for speed and concentration. These findings can also be viewed through the lens of technology acceptance model (TAM) research [11], where factors such as perceived usefulness influence users' willingness to adopt and rely on new technologies.

Visual attention patterns also differed by assistance type. Although most gaze time was focused on the code pane in both conditions, Copilot participants spent slightly more time on the Copilot window and shifted their attention more frequently and for longer periods to the project structure and stack trace panes. These differences may suggest variations in engagement and problem-solving approaches when using different types of assistance.

We make the following contributions:

- **Empirical comparison of AI and human assistance:** A controlled study with 50 participants comparing GitHub Copilot and peer assistance using objective and subjective measures.
- **Insights into visual and cognitive interaction patterns:** Evidence of how assistance type shapes cognitive load and visual engagement.
- **Preliminary design implications:** Initial considerations for SE tools and education, including attention-aware interfaces and training for critical AI use.

II. Related Work

A. AI-Assisted Versus Human-Assisted Programming

With the growing integration of AI-powered tools and LLMs into everyday development practices, there is a pressing need to examine how developers engage with these tools.

Previous research has assessed the quality of code and explanations produced by LLMs while examining their impact on programming and computing education [25]–[27], and explored their potential integration into development tools [28], [29].

Several studies have investigated how AI-assisted tools perform in practice, using real user interactions or behavioral data, with some focusing on GitHub Copilot [13]–[15], [25], [30]. Barke *et al.* [15] identified two modes of developer interaction with Copilot—*acceleration*, where users swiftly complete code they already have in mind, and *exploration*, where users deliberately probe options to guide uncertain steps. These modes are fluid and context-driven, suggesting future assistants should adapt accordingly. Similarly, Prather *et al.* [31] found that novice programmers using Copilot often engaged in two distinct interaction patterns—*shepherding*, where they shaped suggestions toward desired outcomes, and *drifting*, where they followed incorrect suggestions without clear direction. These behaviors highlight Copilot's dual potential to scaffold learning and to induce confusion, especially when students over-rely on generated code without fully understanding it. This confusion was also previously noted by Vaithilingam *et al.* [32], who found that while developers appreciated Copilot's ability to provide useful starting points, they often struggled to understand or debug its outputs. These difficulties sometimes led users into time-consuming "debugging rabbit holes," suggesting future tools should offer better support for validating and explaining generated code. Shah *et al.* [33] further expanded on this by introducing the notion of *one-shot prompting*, where students prompt Copilot to generate an entire feature at once, followed by iterative debugging. These behaviors illustrate an emerging challenge: while Copilot can accelerate certain tasks, it often demands high cognitive vigilance to verify and interpret its output—vigilance that can give rise to the very confusion and misdirection previously observed by Vaithilingam *et al.* [32] and Prather *et al.* [31].

Ma *et al.* [34] compare peer and AI pair programming, examining their similarities, differences, proposed benefits, and associated challenges. They summarize key factors that influence successful collaboration, while acknowledging that the evaluation measures used in the literature for traditional pair programming are more comprehensive. Moreover, they highlight that one recurring failure in peer programming is the free-rider problem, where the bulk of the workload falls on one participant. A common remedy in human-human pairs is regular role-switching, allowing both individuals to alternate between the driver (code writing) and navigator (code suggestion) roles. In human-AI pair programming, tools like Copilot naturally assume the driver role due to their rapid code generation. As a result, it is recommended that the human reclaim the agency and alternate between both modes of engagement. This mimics the collaborative dynamics of human pairs and helps shift part of the cognitive burden—from passively verifying AI-generated code to actively writing with the AI. Such a balance ensures the human remains cognitively engaged and avoids falling into a passive, free-rider-like position.

Prior work largely relies on qualitative observations, self-reports, or post-task outcomes, leaving real-time cognitive behaviors underexplored. Moreover, many evaluations compare AI assistance only to unassisted work and often rely on small code snippets, limiting real-world validity. This study extends prior work by examining AI-assisted navigation within a mature codebase while analyzing gaze-based strategic behaviors in AI versus human programming contexts.

B. Trust and Reliance in Human-Automation Interaction

Previous research on the use and adaptation of AI-powered tools like GitHub Copilot in software engineering has shown improvements in developer speed and satisfaction, while also raising concerns about overreliance and the need for trust calibration [13], [15], [35].

Importantly, trust in AI-generated suggestions has been shown to vary by feature. In Shah *et al.*'s study [33], students expressed greater trust in Copilot's comprehension features (e.g., code explanation) than in code generation, potentially due to "trust affordances" such as clickable code references and transparent reasoning. Such asymmetries in trust suggest that designers should consider interface features that support user confidence without encouraging blind reliance. A handful of prior studies have explored the impact of AI code assistants on security-related tasks. While Perry *et al.* [35] found that participants using AI assistants produced less secure code and were more likely to overestimate its security, Sandoval *et al.* [36] observed that such tools do not necessarily introduce new security vulnerabilities. Nonetheless, these findings suggest that AI code assistants should be used with caution.

Recent work also uses gaze data to infer trust and reliance behaviors in human-AI collaboration [20], [21], [25], [37]. Wu *et al.* [21] and Cao *et al.* [20] argue that gaze-based metrics can help quantify reliance on AI suggestions beyond binary outcome-based metrics. Passi *et al.* [37] further caution against over-trusting AI systems when users defer to incorrect outputs, especially in high-stakes decision-making. Wang *et al.* [25] conducted a controlled experiment with 109 participants to evaluate the usefulness of ChatGPT in coding. They found ChatGPT helpful for simple coding tasks but less effective for typical software development. Participants also showed lower trust in the AI and paid closer attention to prompt phrasing, leading to more consistent responses.

Despite advances in understanding trust, reliance, and human–AI interaction, these topics remain largely disconnected from cognitive measurement. Prior studies focus on performance or perceptions but rarely analyze gaze-based cognitive behavior in AI versus human programming contexts. This study addresses this gap through a within-subjects comparison with a human-peer baseline, allowing us to examine whether observed attention and reliance patterns are specific to AI assistance or reflect general collaborative problem-solving.

C. Cognitive Load and Visual Attention in Programming

Cognitive load in software engineering reflects the mental effort required to comprehend, write, and maintain code. Foundational theories, such as the eye-mind hypothesis, establish that gaze metrics—particularly fixation durations and counts—serve as reliable proxies for this internal mental

effort [17], [23], [38]. Recent literature suggests that AI assistants can accelerate task completion, but they also introduce a verification tax, shifting the developer's effort from generative problem-solving toward code comprehension and validation [14]. However, it remains unclear whether the cost of verifying AI-generated suggestions differs from evaluating a human peer's reasoning during collaborative programming.

Beyond overall mental effort, visual attention patterns reveal developers' information-seeking and problem-solving strategies. Eye-tracking studies show that developers allocate attention differently depending on the provenance of code. For example, developers exhibit distinct visual scanning behaviors when reviewing AI-generated patches compared to human-authored ones, often shifting attention toward secondary verification artifacts such as tests or documentation [18], [19].

Recent work also shows that validating LLM-generated code involves complex, non-linear navigation strategies within the integrated development environment (IDE) [10]. However, prior research has largely examined these behaviors in static review settings or isolated AI usage.

Building on these insights, our study examines real-time cognitive load and visual attention during active programming tasks. By directly comparing GitHub Copilot assistance with collaboration with a human peer in a within-subjects design, we analyze how different assistance types shape cognitive load, attention patterns, and reliance behaviors.

III. Experimental Methodology

This section describes the experimental protocol for our eye-tracking study. Our study was conducted under the oversight of an institutional review board, and informed consent was obtained from all human participants prior to their involvement in the study. Study materials, including stimuli and de-identified data, are available on the project website.[1] In accordance with our IRB approval, raw data cannot be made publicly available online but may be shared privately with interested researchers upon request.

A. Overview of the Study Design

We conducted a within-subjects, in-person experiment to examine how the type of assistance—either a human peer or GitHub Copilot (our core independent variable)—influences developers' performance, cognitive load, reliance, trust, and visual interaction strategies during software development tasks. In this eye-tracking study, all 50 participants were tasked with completing two sets of debugging and comprehension tasks for approximately 20 minutes each.

B. Participants and Recruitment

A total of 50 participants were recruited, primarily undergraduate and graduate students in software engineering, including some who had recently completed their bachelor's degrees. These recent graduates are labeled as B. Grad. in Table I. Applicants from other engineering backgrounds were included only if they had experience with object-oriented programming and Python. With regard to programming experience, participants reported an average of 4.87 years, with most falling in the 3–7 year range and a median of 5 years. For Python experience, participants rated themselves on a scale from 1 (very inexperienced) to 10 (very experienced), with an average of 4.09 and a median of 3.0.

TABLE I
Overview of participant demographics, including age, gender, study level, and Python experience.

Characteristics	All ($N = 50$)	Men ($n = 41$)	Women ($n = 8$)	Prefer not to respond ($n = 1$)
Age (n, %)				
18–25	34 (68.0%)	27	6	1
25–30	13 (26.0%)	12	1	0
30–35	3 (6.0%)	2	1	0
Study level (n, %)				
Freshman	11 (22.0%)	9	2	0
Sophomore	2 (4.0%)	1	1	0
Junior	7 (14.0%)	5	1	1
Senior	8 (16.0%)	8	0	0
B. Grad.	7 (14.0%)	6	1	0
M.S./Ph.D.	15 (30.0%)	12	3	0
Python experience (mean ± std dev)				
Freshman	5.8 ± 1.0	5.7 ± 1.0	6.5 ± 0.7	–
Sophomore	4.0 ± 2.8	6.0 ± –	2.0 ± –	–
Junior	5.7 ± 1.7	5.4 ± 1.7	8.0 ± –	5.0 ± –
Senior	7.2 ± 0.9	7.2 ± 0.9	–	–
B. Grad.	4.6 ± 1.7	4.7 ± 1.9	4.0 ± –	–
M.S./Ph.D.	6.7 ± 2.7	6.8 ± 2.7	6.3 ± 3.1	–

Note: "–" indicates either there were no participants in that subgroup (n = 0) or the standard deviation is not reported because there was only one participant (n = 1).

There is no universally accepted standard for categorizing novice developers as the title varies across companies, industries, and regions [39]. Importantly, these categories are typically defined in terms of years of professional practice in the field excluding years of learning or academic study. Since our participants were students with little to no professional experience, we define novices in terms of the self-reported programming experience and Python proficiency mentioned previously. On this basis, we classify our participants as novice developers.

Regarding prior experience with AI tools, 96% of participants reported having used them—many on a daily basis (29 participants) or weekly basis (7 participants). Notably, 32 participants reported using GitHub Copilot in their regular development tasks, particularly for code generation and debugging.

The screening process for each applicant was conducted through a pre-task questionnaire, which also collected demographic and technical background information meant to determine whether they met the inclusion criteria.

C. Software Systems and Tasks

To reflect real-world scenarios, the programming tasks were drawn from issues reported in the *Glances* GitHub repository. *Glances* is an open-source system monitoring tool written in Python, offering functionality similar to popular utilities such

[1] https://github.com/SENSE-Research-Lab/Trust-Reliance-AI-Coding

as `top` and `htop`. We focused on tasks involving common feature enhancements, in line with typical software maintenance practices, requiring code comprehension and active debugging. Each task involved modifying or adding code and was designed to reflect one of the core themes outlined in Table II (labeled T1-T6), such as filtering, logging, and bug fixing. All tasks were carried out using the PyCharm IDE in a controlled lab environment.

GitHub Copilot was selected as the AI assistant in this study given its widespread adoption. As of October 2024, Copilot has been adopted by over 77,000 organizations, including more than 90% of Fortune 100 companies [40].

Each participant completed two sets of programming tasks—one set assisted by GitHub Copilot and the other by a human peer—with each set consisting of three tasks (six tasks in total). To minimize task-specific bias, the assignment of tasks to each set was randomized individually per participant, ensuring that any given task could appear in either set.

Task randomization was handled by a Python script executed prior to each participant's arrival. This script generated a personalized `instructions.txt` file containing the six tasks distributed across the two assistance conditions, along with instructions on how to launch and interact with the *Glances* application.

Pilot testing with proficient developers confirmed that each set of tasks could be completed in approximately 20 minutes, a duration that was also feasible for novice participants when assisted by GitHub Copilot or a peer.

D. Equipment and Setup

We used the Tobii Pro Fusion eye trackerwith a sampling rate of 250 Hz to capture participants' gaze data throughout the experiment. Before each set of tasks, Tobii Eye Tracker Manager was used to guide participants into the correct seating position using a visual head position indicator. Calibration was then performed using Tobii Pro Lab, which also recorded the screen during task execution. The screen used was a Dell 24 - P2422HE with USB-C hub, a 23.8-inch monitor (diagonal) with a native resolution of 1920 x 1080 pixels, and a refresh rate of 60 Hz. To preserve the integrity of predefined Areas of Interest (AOIs) for eye-tracking analysis, participants were not permitted to rearrange or resize any windows once the calibration was finished. Since AOIs are defined by fixed screen coordinates, any changes to window layout would invalidate gaze-to-AOI mappings and compromise the consistency of the data.

E. Procedure

In the Copilot phase, participants interacted with GitHub Copilot using the official plugin available in PyCharm via the JetBrains Marketplace. In the Peer phase, to ensure that the interaction experience between the Peer and AI (Copilot) conditions remained as consistent and comparable as possible, we opted for a controlled, text-only environment. SimpleX Chat was deliberately selected over richer communication platforms (e.g., Zoom), which could introduce variability in tone, presence, or communication dynamics. For example, it is easier for two different experimenters to minimize variability when interacting through instant messaging than when conversing in real time via audio or video. By standardizing both conditions to text-based interaction, we maintained a fair comparison across experimental phases. In addition, it was chosen for its straightforward setup and scalability; it allowed the creation and management of 50 locally stored user accounts without requiring email or phone authentication.

One of the experimenters acted as the peer assistant, following a guideline to avoid directly providing solutions and instead encourage problem-solving through conversational support, while maintaining consistency across sessions. Participants were informed that they would be interacting with a peer developer familiar with the codebase, but were not made aware that the peer was in fact a researcher. This approach was intended to mitigate bias linked to perceived authority or evaluation, while still providing participants with a credible and supportive collaborator.

Participants were free to complete the tasks in any order and could skip any questions if desired. Upon completing both phases, they answered a post-task questionnaire measuring trust, reliance, task recall, and subjective preferences regarding the two assistance modalities. Each session lasted approximately 60 minutes, including setup, calibration, task completion, and post-task questionnaires.

F. Eye-tracking Data Preparation and Analysis

Eye-tracking data was categorized into fixations and saccades [22]. A fixation refers to a relatively stable gaze lasting approximately 200–300 milliseconds, and plays a critical role in information acquisition and cognitive processing [24], [38]. In contrast, saccades are rapid eye movements between fixations and involve minimal cognitive processing.

We used Tobii Pro Lab to capture participants' eye movements and compute standard fixation-based eye-tracking metrics for assessing cognitive load and visual effort [23], [24]. We analyzed participants' gaze behavior using predefined Areas of Interest (AOIs) within the IDE. Following AOI design guidelines by Goldberg and Helfman [41], we manually segmented each interface into four rectangular AOIs: the project structure (AOI 1), code pane (AOI 2), stack trace (AOI 3), and assistance section which could either be the Copilot pane (AOI 4A) or the SimpleX chat window (AOI 4B). These AOIs were consistently sized, placed, and remained visible throughout the experiment as shown in Figure 1.

G. Experiment Measures

We evaluated participants' task performance, their visual attention dynamics, and the degree to which they relied on the provided assistance. Table III provides definitions for all evaluation metrics.

Performance: To evaluate participants' performance, we used three proxies: 1) cognitive load, as evidenced by eye-tracking data, which provided insights into participants' level of engagement; 2) task time, defined as the total time each participant spent completing the tasks, offering a measure of task efficiency; and 3) accuracy, measured as an ordinal

TABLE II
OVERVIEW OF PROGRAMMING TASKS ASSIGNED DURING THE STUDY, INCLUDING TASK DESCRIPTION, ASSOCIATED FILE, AND TASK THEME.

Task Description	Associated File	Task Theme
T1: Create a `PluginStatistics` class to report active/disabled plugins using `getPluginsList`.	`glances/stats.py`	**Feature Implementation:** Introduces a new class leveraging existing APIs to report runtime plugin states.
T2: Add a `serve_limited` method to `GlancesXMLRPCServer` that stops after a set number of requests.	`glances/server.py`	**Shutdown Logic + Logging:** Adds a request-bound execution loop requiring termination logic and logging.
T3: Modify `getAll` to support filtering plugins by name via an optional `filter_list` parameter.	`glances/stats.py`	**Conditional Data Filtering:** Enhances a core method to support optional input-driven filtering logic.
T4: Add logging in `check_user` to record usernames and authentication outcomes.	`glances/server.py`	**Security + Logging:** Instruments authentication flow with structured logging based on outcomes.
T5: Add timestamp to fallback logic in `get_io_counters` for when defaults are used.	`glances/processes.py`	**Fallback Enhancement:** Improves fallback path with diagnostic timestamping to aid traceability.
T6: Update `update_processcount` to handle edge cases of invalid/missing process statuses.	`glances/processes.py`	**Bug Fixing:** Corrects faulty use of `is` versus `==`, improving logic reliability.

TABLE III
SUMMARY OF PERFORMANCE, ASSISTANCE RELIANCE, AND VISUAL ATTENTION METRICS.

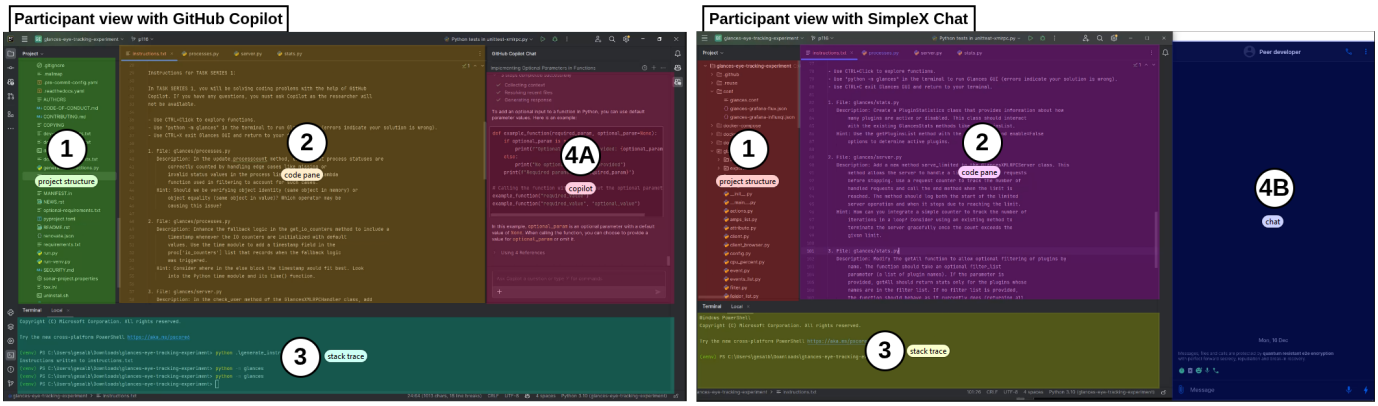
Performance Metrics	Definition
Cognitive Load	**Fixation Count:** Total number of fixations on the stimulus during the task. Provides insight into how visual attention is distributed and how efficiently relevant information is located. Fixation count reflects the intensity of visual search and information acquisition. Higher counts indicate more exhaustive searching effort or difficulty integrating information [22], [24] (e.g., reconciling AI suggestions with existing code), whereas lower counts may reflect efficient navigation or attentional tunneling toward a narrow solution path.
	Average Fixation Duration: Average duration of all fixations during the task. Longer durations may reflect increased cognitive load or difficulty in processing information [41], [42]. Average fixation duration maps the depth of information processing. Longer durations typically signify greater difficulty in extracting meaning from a stimulus or resolving ambiguity (e.g., interpreting complex stack traces or unfamiliar logic) [38].
	Total Fixation Time: Total time spent fixating on the stimulus indicates visual engagement and the cognitive load required to process and extract information. Total fixation time maps cumulative visual engagement. When analyzed alongside task time, it serves as a joint proxy for the total cognitive budget allocated to a specific AOI.
Task Time	Total time each participant spent completing the tasks, offering a measure of task efficiency.
Accuracy	Extent to which the solution satisfies three evaluation criteria—code correctness, successful compilation, and correct output—with scores ranging from 0 to 3 (0%, 33%, 66%, or 100%).
Assistance Reliance Metrics	**Definition**
Perceived Reliance	Participants' subjective ratings of trust in AI tools and the extent to which they relied on suggestions from either AI or a peer, measured using a 5-point Likert scale (1 = Not at all, 5 = Drastically).
Assistance Attention Ratio	Proportion of time a participant spent fixating on the assistance window relative to total fixation time across all task-related areas of interest (AOIs) [20], [21], [37].
Number of Visits	It captures how often participants redirected their gaze to the chat or Copilot panes after looking elsewhere, indicating recurring attention and potential uncertainty or confusion [21].
Visual Attention Metrics	**Definition**
Gaze Allocation	By using fixation count and duration, it assesses attention distribution and navigation across interface areas during the task.

score reflecting the extent to which the participant successfully completed the task.

We operationalize cognitive load using eye-tracking metrics that capture visual attention during task execution. Fixation duration reflects the time spent processing a single element, while fixation count and total fixation time capture the distribution and overall amount of visual attention across IDE panes. These metrics allow us to quantify how participants allocate attention when interacting with different assistance types.

Assistance reliance: Drawn from prior research [20], [21], we employed three metrics to evaluate developers' reliance on the proposed assistance: 1) self-reported perceived reliance, 2) Assistance Attention Ratio, and 3) number of visits to Copilot or chat pane. Previous research has demonstrated a strong correlation between participants' agreement with the assistance's suggestions and the duration of their visual attention on the AI's input [20], [21].

Visual attention: We compute fixation count and total fixation time across all areas of interest (e.g., code pane, assistant panes, project structure, and stack trace) and analyze their distribution to examine how different types of assistance shape participants' visual attention patterns. These fixation-



(1) Project Structure: Displays the hierarchical organization of files in the Glances project. Participants could refer to it to navigate across modules, directories, or packages.

(2) Code Pane: The central workspace where participants read, modified, and wrote Python code to complete the programming tasks. Participants also used it to consult the generated text file containing the instructions for their tasks.

(3) Stack Trace: The output window that showed terminal messages, including errors and runtime feedback. Participants used it to diagnose issues and verify execution.

(4A) Copilot Pane (Only with GitHub Copilot): Displays GitHub Copilot's chat interface. Participants interacted with this pane to request help and evaluate AI-generated code completions.

(4B) Peer Chat Window (Only with SimpleX Chat): Used for peer assistance, this pane hosted real-time textual communication with a human experimenter acting as a peer developer. Participants used it to seek guidance or ask clarifying questions.

Fig. 1. Participant view across the two assistance conditions with defined Areas of Interest (AOIs). Left: GitHub Copilot view with (1) project structure, (2) code pane, (3) stack trace, (4A) Copilot chat. Right: SimpleX Chat view with the same three AOIs (1–3) and (4B) peer chat.

based metrics capture how gaze is allocated across interface elements and how participants navigate the environment during task completion [8], [19].

While fixation count reflects how frequently an AOI is visited, fixation duration indicates how long gaze is maintained during a visit, and total fixation time represents the cumulative attention devoted to an AOI across the task. Visual attention is treated as a behavioral construct distinct from cognitive load and reliance. While fixation activity may reflect cognitive load, visual attention captures *where* participants focus and *how* they navigate the interface. Cognitive load is interpreted alongside task difficulty and performance, whereas reliance is inferred from attention directed toward the assistance panes, enabling separate analysis of interaction strategies, and reliance behavior.

IV. Data Analysis and Results

In this section, we examine how participants performed and interacted with the two types of assistance. For each metric, we report the means and standard deviations. A Wilcoxon signed-rank test ($\alpha = 0.05$) was used to assess whether differences between the two conditions were statistically significant. Effect sizes are reported using Cohen's d. We also include qualitative insights from participants regarding their experiences with, reliance on, and interactions with AI assistance.

A. RQ1. Assistance Type and Performance

We investigate the effect of assistance type on participants' performance, specifically in terms of task duration, accuracy, and perceived cognitive load.

As presented in Table IV, participants spent significantly more time on the task when working with Copilot (17.9 minutes) compared to when working with a peer (16.8 minutes), averaging approximately 6.6% more time.

TABLE IV
PAIRWISE COMPARISONS OF PERFORMANCE AND RELIANCE METRICS WERE CONDUCTED USING THE WILCOXON TEST ($\alpha = 0.05$) TO ASSESS DIFFERENCES BETWEEN ASSISTANCE TYPES (COPILOT VS. PEER). SIGNIFICANT RESULTS ($p < 0.05$) ARE SHOWN IN BOLD. PARTICIPANTS USING COPILOT SPENT MORE TIME ON THE TASK, EXHIBITED HIGHER COGNITIVE LOAD AND VISUAL EFFORT, AND DEVOTED MORE FIXATION TIME TO THE COPILOT CHAT, INDICATING GREATER RELIANCE, WHILE PEER CHAT INTERACTIONS WERE SHORTER BUT MORE FREQUENT.

	Copilot	Peer	p-value	W	Cohen's d
Performance					
Task time (min)	17.9 (3.8)	16.8 (2.09)	**0.003**	198.0	0.33
Avg. Fx. Dur (ms)	319.3 (61.8)	292.9 (63.7)	0.05	354.5	-
Fx. Count	1763 (413)	1545 (438)	**0.01**	293.0	0.51
Total Fx. Time (s)	584.3 (142.7)	505.6 (141.2)	**0.03**	258.0	0.55
Reliance					
Attention Ratio	0.085 (0.05)	0.063 (0.04)	**0.045**	1288	0.45
Visit Number	29.8 (19.7)	40.8 (22.9)	**0.018**	773	0.51

Their accuracy was also significantly higher—50% compared to 32%—when interacting with Copilot rather than a peer. A binomial generalized estimating equation model [43] revealed a significant effect of assistance type on accuracy ($z = -2.71$, $p = 0.007$), with lower odds of correct responses in the human-peer condition relative to Copilot. Correspondingly, accuracy was higher under Copilot assistance, with a medium effect size ($d = 0.55$).

In terms of cognitive load, participants in the Copilot condition demonstrated significantly higher load. Specifically, they exhibited a higher number of fixations and spent more total time fixating on the screen compared to those in the peer condition. This suggests that working with Copilot may demand greater visual attention or cognitive processing during task completion.

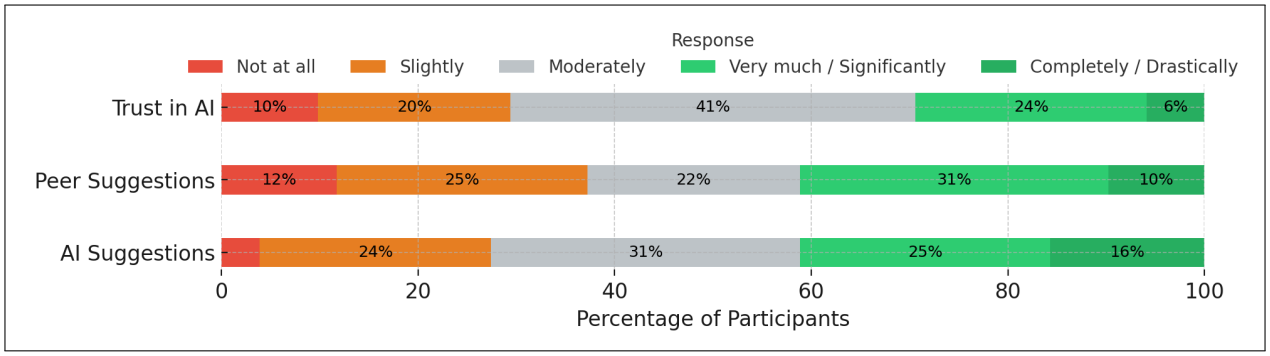


Fig. 2. Participants' self-reported ratings of their reliance and trust across three aspects: Trust in AI, Reliance on Peer Suggestions, and Reliance on AI Suggestions. Responses were collected on a 5-point Likert scale ranging from 1 (Not at all) to 5 (Completely/Drastically). The majority of participants reported moderate to high reliance on both AI and peer suggestions. Trust in AI was also moderately high, with 71% of participants rating their trust as 3 or above.

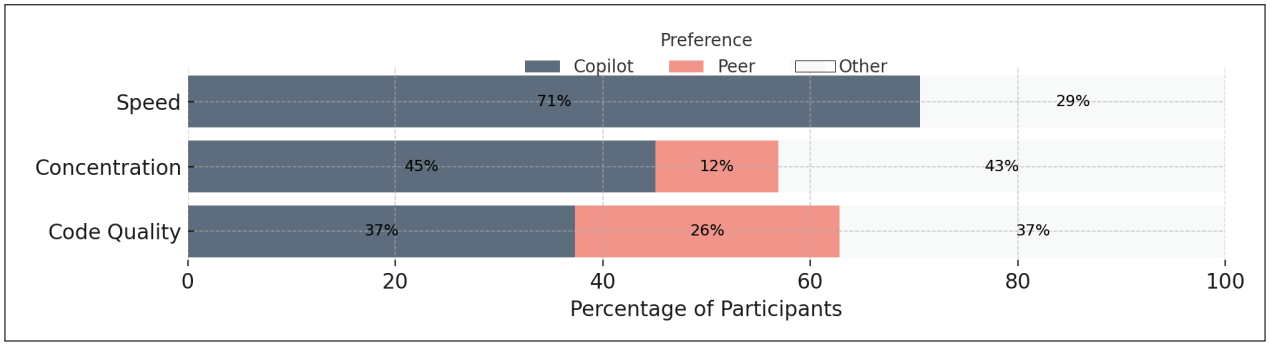


Fig. 3. Participant preferences regarding the perceived impact of assistant type (Copilot vs. Peer) on Speed, Concentration, and Code Quality. While a strong majority (71%) reported that GitHub Copilot improved their speed, preferences for concentration and code quality were more varied. Copilot was still favored for concentration (45%), though a substantial portion selected "Other" responses (43%), indicating nuanced or context-dependent experiences. For code quality, preferences were more evenly distributed across Copilot (37%), Peer (26%), and Other (37%), suggesting no clear consensus.

B. RQ2. Assistance Type and Reliance

Pairwise comparisons of the reliance metric (Gaze Allocation), using a non-parametric Wilcoxon test ($\alpha = 0.05$) for assistance type (Copilot vs. Peer), revealed a statistically significant difference between the two conditions, as shown in Table IV. On average, participants spent 8.5% of their time interacting with Copilot, compared to 6.3% when chatting with a peer. However, participants made significantly more visits to the human peer's chat window than to the Copilot chat window, suggesting that human interaction may have encouraged more frequent check-ins or back-and-forth engagement. This pattern reflects a more distributed interaction style with the peer, whereas interactions with Copilot tended to be longer but less frequent.

We also conducted a qualitative analysis of participants' self-reported data. The results for the following three questions are summarized in Figure 2, using a 5-point Likert scale ranging from 1 (Not at all) to 5 (Drastically).

1) How much do you trust the output generated by AI tools?
2) To what extent did you rely on your peer's suggestions in your tasks?
3) To what extent did you rely on the AI's suggestions in your tasks?

Regarding daily use of AI-powered tools and LLMs, trust most frequently peaks at Moderately and Significantly. In the context of the study, participants reported relying more heavily on peer suggestions at the high end (Very much and Completely) compared to AI. AI-generated suggestions were more commonly used at the Slightly or Moderately levels.

Moreover, in the post-task questionnaire, we asked participants about their preferences and how they perceived the assistant type to impact their speed, concentration, and code quality in daily tasks. The results are summarized in Figure 3. The findings show that:

- Speed: A strong majority preferred GitHub Copilot.
- Concentration: Copilot was favored, though responses categorized as Other (including "No difference," "Depends," and "Unclear") were also substantial.
- Code Quality: Responses were more mixed, with nearly equal preference among Copilot, Peer, and Other.

C. RQ3. Assistance Type and Visual Attention

We calculate the fixation metrics—fixation count and fixation time—within each AOI to determine whether a difference exists between the attention distribution of participants while interacting with a peer or Copilot. We use a general align-and-rank non-parametric factorial analysis [44]. We find that there is a significant interaction between assistance type and AOIs: $F(2, 311.23) = 6.18$, $p = 0.002$ for fixation count and $F(2, 311.23) = 5.12$, $p = 0.0006$ for fixation time. This interaction is reflected in Figure 4, where the distribution of normalized fixation durations differs markedly between Copilot and Peer conditions depending on the AOI.

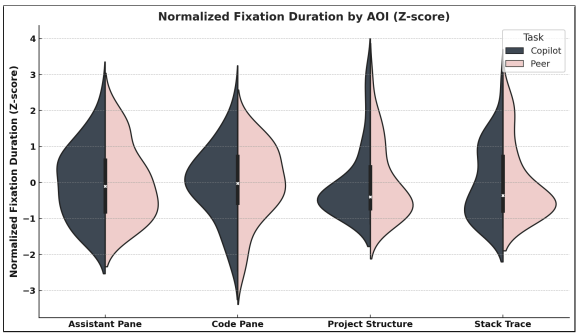


Fig. 4. Z-score normalized fixation time across four Areas of Interest (AOIs): Assistant Pane, Code Pane, Project Structure, and Stack Trace. Each violin shows the distribution of visual attention (in Z-scores) for participants completing tasks with GitHub Copilot and with a human peer. The left side of each violin represents Copilot, and the right side represents Peer.

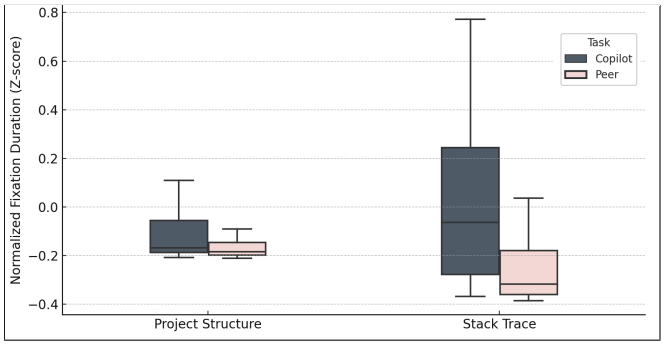


Fig. 5. Distribution of fixation times across Stack Trace and Project Structure panes between two assistance types. Copilot is associated with higher relative fixation durations in the Stack Trace and Project Structure panes, suggesting deeper visual engagement in those interface elements.

Differences in the Assistant Pane and Code Pane were less pronounced. In the case of Copilot, the code pane accounted for 80.6% and the Copilot window for 14.0% of the visual attention, while for the Peer assistance type, the code pane slightly increased to 83.3% and the chat window to 14.8%.

However, participants spent relatively more time fixating on the Stack Trace and Project Structure panes when using Copilot, as shown in Figure 5, demonstrating assistance-type-dependent variation in how developers engaged with these AOIs. The difference in visual attention was statistically significant for Stack Trace ($W = 1515$, $p < 0.001$) and marginally significant for Project Structure ($W = 1159$, $p = 0.05$). These results suggest that the type of assistance modulates how participants allocate visual attention across interface components. These differences likely reflect distinct interaction mechanisms between AI and human assistance. With Copilot, participants may have engaged in additional verification of generated suggestions, explaining the increased attention to diagnostic elements such as the stack trace and project structure. In contrast, collaboration with a human peer involved conversational exchanges, leading to more frequent attention shifts toward the chat interface.

V. Discussion

Our findings reveal important trade-offs between AI-assisted and human-assisted programming. We now elaborate on each of our research questions.

RQ1. *Assistance Type and Performance*: Both our quantitative and qualitative findings indicate that the type of assistance influences task outcomes in terms of efficiency, accuracy, and engagement. Participants working with Copilot spent approximately 6.6% more time completing tasks compared to those paired with a peer, but also achieved significantly higher accuracy. In terms of cognitive load, Copilot participants demonstrated greater engagement, reflected by a higher number of fixations and longer total fixation time. These findings suggest that while Copilot may require more mental effort, it is associated with better task accuracy.

Prior work, particularly in industrial settings [45], often reports substantial efficiency gains from AI-assisted programming, with controlled studies showing large reductions in task completion time for well-defined, boilerplate-heavy tasks. In contrast, our findings reveal a different trade-off in an educational context. Although Copilot use was associated with higher accuracy, students required more time and exhibited greater cognitive load than those working with a peer. This suggests that, for student programmers and smaller-scale tasks, AI assistance may shift effort from rapid code production to interpretation, verification, and trust calibration rather than accelerating execution.

Importantly, our results show that increased cognitive load can coexist with improved performance. Participants were willing to invest more time and attention when this effort led to higher task correctness, challenging the assumption that effective programming tools should primarily minimize cognitive load. Instead, Copilot appears to support correctness by channeling effort toward careful reasoning and validation, and participants' preference for AI assistance, despite longer completion times, suggests a tendency to favor outcome quality over speed.

Ma *et al.* [34] also offered a nuanced perspective on the efficacy of human-AI versus human-human pair programming. While outlining the advantages and limitations of both approaches, they emphasized the need for more comprehensive evaluation metrics, a gap our study investigates by combining performance measures with gaze-based behavioral data. Specifically, our findings provide empirical evidence for what Ma *et al.* [34] describe as the need to "co-verify" answers when a human partner is absent. Without a peer to discuss logic or clarify requirements, participants using Copilot were forced to take on a more intensive "navigator" role. This behavioral shift is clearly reflected in our eye-tracking metrics. AI-assisted participants directed significantly more fixations and longer gaze durations toward diagnostic areas, such as the stack trace and project structure panes, in order to validate the generated code.

RQ2. *Assistance Type and Reliance*: Both behavioral and self-reported data suggest that participants exhibited greater reliance on Copilot compared to a human peer. Quantitatively, participants spent a significantly larger proportion of their task time interacting with Copilot (8.4%) than with a peer (6.3%). However, they made more frequent visits to the peer chat window than to the Copilot interface, suggesting different patterns of engagement. Qualitative responses revealed that

participants generally trusted AI tools at a moderate level. When asked about perceived impacts on performance, most participants preferred Copilot for improving speed and concentration. However, opinions on code quality were more evenly split among Copilot, peer assistance, and neutral responses.

These patterns are also consistent with technology acceptance research, which suggests that perceived usefulness shapes users' willingness to adopt and rely on new technologies [11]. Participants' preference for Copilot in terms of speed and concentration may reflect this perceived usefulness, while the more mixed responses regarding code quality suggest that acceptance depends on perceived reliability and task context.

Participants' longer but less frequent interactions with Copilot are consistent with follow-up oriented behavior in which the AI assistant is consulted repeatedly for clarification and refinement. This interaction style aligns with participants' self-reported moderate trust in AI tools, which suggests that Copilot was treated as a quick responsive resource rather than an authoritative source of information. In contrast, the higher number of visits to the peer chat window reflects the conversational dynamics of human textual interaction where shorter and more frequent exchanges are common.

Participants' preference for Copilot for speed likely reflects its immediacy. Unlike human peers, Copilot provides instant responses without delays. This enables rapid iteration and uninterrupted task flow. Importantly, this preference for immediacy did not translate into uniformly higher confidence in code quality. Participants' mixed evaluations of Copilot's impact on code quality indicate a selective calibration of trust. Copilot is perceived as advantageous for maintaining momentum but not consistently superior for producing higher-quality solutions. Together, these findings suggest that programmers strategically leverage AI assistance for fast feedback while remaining cautious about its outputs.

Viewed through the lens of calibrated reliance [7], participants did not appear to rely on the AI uncritically. Instead, their interaction patterns suggest continued monitoring of AI outputs. Although the tool may support rapid suggestions, participants still devoted attention to reviewing and verifying them, indicating that AI assistance may shift effort toward evaluating generated code.

RQ3. *Assistance Type and Visual Attention*: Participants working with Copilot directed most of their attention to the code pane and the Copilot window, while those in the Peer condition showed a slightly greater focus on the code pane and chat window. We also observed noticeable differences in how participants engaged with the Stack Trace and Project Structure panes depending on the assistance type, indicating that AI and peer support guide visual navigation differently during task completion.

Taken together, these patterns suggest that the form of assistance may influence how participants allocate visual attention during programming tasks. Under AI assistance, participants devoted more attention to diagnostic and code-related regions of the IDE, consistent with increased inspection of generated suggestions. In contrast, when working with a human peer, attention was directed more toward the chat interface. These differing gaze patterns demonstrate that substituting a human peer with an AI assistant shifts the developer's core debugging strategy. While human peers facilitate contextual understanding through dialogue, AI tools require the developer to independently hunt for diagnostic proof.

Our findings align with prior work showing that the source modality—whether code or assistance originates from a human or a machine—shapes participants' visual attention and interaction patterns [8], [18], [25], with AI-based assistance, in particular, leading to more dynamic and engaged interactions [10], [25].

A. Implications for Research

Our study expands current understanding of how assistance type influences cognitive processes, reliance behaviors, and visual attention dynamics in software development, more specifically debugging. By combining eye-tracking data with performance metrics, we demonstrate the value of multimodal methods for capturing developers' real-time engagement with AI and human collaborators.

Notably, our findings show that visual attention data—such as fixation time and frequency—can serve as a meaningful proxy for measuring reliance, offering a more continuous and unobtrusive alternative to traditional self-report or outcome-based measures. Consistent with previous work [20], [37], longer gaze durations on the AI interface indicate greater reliance on the AI assistant. This has important implications for how reliance is conceptualized and assessed in future research. Existing methods typically rely on outcome agreement or subjective perception [21], [37]. While outcome agreement defines reliance as appropriate only when the user makes the correct decision, it fails to distinguish between critical engagement and blind acceptance of AI suggestions. Similarly, self-reported reliance is prone to bias and lacks the granularity to capture moment-to-moment fluctuations. By contrast, our results suggest that eye-tracking data can bridge this gap, providing researchers with a fine-grained, real-time lens into how developers interact with assistance.

Future work should build on this by examining how reliance and attention patterns evolve over time, across varied task types, and with more experienced developers. In addition, researchers might explore how AI tools can be adapted to individual developers by modeling their cognitive load and visual engagement, ultimately leading to more effective and personalized support systems.

B. Implications for Practice and Education

The findings may be useful for practitioners considering how to integrate AI tools such as GitHub Copilot into development workflows or training contexts. While Copilot was associated with improvements in task accuracy and speed, it also appeared to increase visual and cognitive demands. These trade-offs suggest that some form of onboarding or guided practice could be beneficial, particularly for novice users.

More broadly, participants' interaction patterns indicate that increased cognitive load is a natural and productive component of AI-assisted programming. Rather than passively accepting suggestions, participants consistently invested attention in

reasoning about and validating AI-generated code. Because this effort is directed toward verification and understanding, it serves an essential role in effective AI use. As such, onboarding strategies should focus on maximizing the productivity of this validation rather than attempting to reduce load altogether. This is particularly useful in early-stage software development where substantial cognitive load is already required for core software design decisions. The gains in speed afforded by AI assistance could equally complement such work.

Differences in attention patterns also point to potential considerations for interface design. In the Copilot condition, participants engaged more frequently with interface elements such as the Stack Trace and Project Structure panes. Subtle interface features—such as visual cues or lightweight nudges—may help reinforce productive exploration of these components. For example, providing multiple autocomplete options directly within the code pane rather than in a separate window could capitalize on AI's speed while reducing the cognitive overhead added by the increase in output validation. Essentially, this approach would narrow how participants naturally distribute their attention by keeping suggestions close to the code editing context. As such, users could rapidly compare alternatives, validate outputs, and integrate them without breaking focus by repeatedly switching between the code, Copilot, and diagnostic panes, which can be cognitively demanding and disrupt flow. This method would also help manage verbose AI responses, which are often skimmed through by users. This design leverages AI's speed while supporting the verification-heavy strategies participants already employ by reducing unnecessary gaze shifts and making the overall interaction more efficient.

Trust in suggestions emerged as another area of distinction: Copilot was preferred for speed and concentration, while its influence on code quality was less consistent. This suggests that promoting transparency and explainability in AI-generated suggestions may support better trust calibration and decision-making. From an educational perspective, our findings indicate that students already exhibit calibrated trust behaviors, including cautious acceptance of AI suggestions, verification-oriented attention, and selective reliance rather than overreliance. This suggests that novice programmers are not uniquely vulnerable to AI misuse or erroneous suggestions. Future work could examine how instruction can further refine these emerging judgment strategies when students encounter competing AI recommendations.

Additionally, the observed patterns of reliance, trust calibration, and verification suggest that AI tools reshape student programmers' cognitive strategies rather than merely accelerating their task execution. These results also have implications for software engineering education. As AI-powered tools become integrated into development workflows, instruction may benefit from explicitly addressing when and how such systems should be used. Helping students understand the strengths and limitations of AI tools, along with practices such as effective prompting, verification, and iterative refinement, may better prepare them for hybrid human–AI development environments. Future research could examine these dynamics in more open-ended programming contexts, such as software architecture design and code review, where more interactive collaboration may shape reliance behaviors differently.

VI. Limitations

To minimize instrument bias, we used a video-based eye-tracker that does not require restrictive goggles and allows for natural head movement without disrupting calibration. The system was carefully calibrated at the start of the study and adjusted between both sets of tasks to maintain accuracy. To mitigate the Hawthorne effect, we clarified that the eye tracker captured only eye-related data, and the experimenter remained seated discreetly at a distance to reduce the sense of being observed. Finally, to avoid stereotype threat and its potential impact on performance [46], [47], participants completed self-assessments of their coding skills only at the end of the study.

The majority of participants in our study were students with solid programming skills. Using student participants is appropriate when the goal is not to investigate the impact of expertise, as they reasonably represent the next generation of software professionals in the software industry [48]. The use of a single subject system may pose a potential threat to validity, as it limits the generalizability of our findings. However, we selected an open-source project implemented in a widely-used programming language, supported by an active developer community, and drew bugs directly from its repository to ensure relevance to real-world software issues.

In our study, we used a relatively coarse-grained outcome measure—whether or not the participant successfully fixed the bug—as the primary measure of accuracy. While this provided a clear and objective way to compare performance across the two assistance conditions, it simplifies the complexity of debugging outcomes. For instance, it does not account for partial progress or the quality of intermediate steps taken toward a solution. Using a more detailed rubric, such as evaluating progress at the sub-goal or code-edit level, could have offered a more nuanced view of participant performance. However, for the sake of reproducibility, we opted for the outcome measure used in this study.

While the majority of our findings are statistically significant, we echo prior research in cautioning against overreliance on significance testing alone [49]. Instead, we emphasize the importance of effect sizes and visual representations to support more nuanced interpretations of the data. Future work with larger samples and deeper qualitative analysis could offer richer insights into participants' visual attention and focus patterns, particularly how they engaged with and adapted to different types of assistance.

A potential threat to internal validity stems from a research team member serving as the human peer assistant and participating in study execution. Although not ideal, participants were informed only that the peer was a classmate who had previously completed the assignment and was familiar with the project. They were not told about the peer's research involvement, minimizing hypothesis guessing and demand effects. The peer did not contribute to the formulation of the research questions, the theoretical framing of the study, or the analysis of the results. During sessions, the peer responded only to participant-initiated questions and provided guidance

without offering complete solutions. The peer was allowed to consult external resources, reflecting realistic collaboration and maintaining parity with AI assistance. While repeated exposure to the tasks may have increased the peer's familiarity over time, using a single peer ensured consistency across sessions. Future work could mitigate this limitation by rotating peers or using larger peer pools.

Finally, to strengthen conclusion validity, we used established eye-tracking metrics, applied standard statistical methods, and calibrated the eye-tracker for each participant before each task set to ensure accuracy and consistency.

VII. Conclusion

As AI-powered tools like GitHub Copilot become increasingly embedded in software development workflows, understanding how developers—particularly novices—engage with these systems is critical. In this study, we conducted a controlled, eye-tracking experiment with 50 participants to compare their interactions with GitHub Copilot and human peer support during programming tasks. Our findings reveal that while Copilot significantly improved accuracy, it also introduced higher cognitive demands, as reflected in gaze behavior and time-on-task measures.

These results suggest that AI assistance does not uniformly replace or surpass human collaboration; rather, it reshapes the way developers allocate attention, process feedback, and manage task complexity. The differences in visual engagement and reliance patterns across assistance types point to the importance of designing future AI tools that balance efficiency with cognitive support. Features that help developers calibrate their trust and reduce switching costs may enhance collaboration and usability.

For researchers, this work contributes empirical evidence that supports the use of multimodal data, including eye-tracking, as a powerful lens into real-time developer cognition and AI reliance. Future research should include a deeper analysis of the chat content between participants and both Copilot and human peers to better understand the nature of their interactions. Long-term work should also extend this study to real-world settings and more experienced developers, examining how adaptive systems can personalize support based on developers' cognitive states. As human-AI collaboration becomes increasingly central to software development, it is crucial to reimagine both development tools and training programs to equip developers with the skills needed to work effectively alongside intelligent assistants.

References

[1] I. Shani. (February 7, 2024) Survey reveals AI's impact on the developer experience. [Online]. Available: https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/

[2] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. J. Jiang, "GitHub Copilot AI pair programmer: Asset or liability?" *Journal of Systems and Software*, vol. 203, p. 111734, 2023. [Online]. Available: https://doi.org/10.1016/j.jss.2023.111734

[3] R. Li, M. Li, and W. Qiao, "Engineering students' use of large language model tools: An empirical study based on a survey of students from 12 universities," *Education Sciences*, vol. 15, no. 3, 2025. [Online]. Available: https://doi.org/10.3390/educsci15030280

[4] W. Sleegers, J. Elsey, and D. Moss, "Adoption and uses of llms among u.s. tech workers," Rethink Priorities, Tech. Rep., July 2025, accessed: 2025-07. [Online]. Available: https://rethinkpriorities.org/research-area/adoption-llms-tech-workers/

[5] G. Fan, D. Liu, R. Zhang, and L. Pan, "The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance: a comparative study with traditional pair programming and individual approaches," *International Journal of STEM Education*, vol. 12, no. 1, p. 16, 2025. [Online]. Available: https://doi.org/10.1186/s40594-025-00537-3

[6] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Human Factors*, vol. 46, no. 1, pp. 50–80, 2004. [Online]. Available: https://doi.org/10.1518/hfes.46.1.50_30392

[7] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Human Factors*, vol. 39, no. 2, pp. 230–253, 1997. [Online]. Available: https://doi.org/10.1518/001872097778543886

[8] S. Yabesi, M. Amini, J. Ristic, and Z. Sharafi, "Exploring the effects of urgency and reputation in code review: An eye-tracking study," in *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, ser. ICPC '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 202–213. [Online]. Available: https://doi.org/10.1145/3643916.3644425

[9] G. M. Alarcon, A. Capiola, I. A. Hamdan, M. A. Lee, and S. A. Jessup, "Differential biases in human-human versus human-robot interactions," *Applied Ergonomics*, vol. 106, p. 103858, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0003687022001818

[10] N. Tang, M. Chen, Z. Ning, A. Bansal, Y. Huang, C. McMillan, and T. J.-J. Li, "Developer behaviors in validating and repairing LLM-generated code using IDE and eye tracking," in *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 2024, pp. 40–46. [Online]. Available: https://doi.org/10.1109/VL/HCC60511.2024.00015

[11] I. Baroni, G. Re Calegari, D. Scandolari, and I. Celino, "AI-TAM: A model to investigate user acceptance and collaborative intention in human-in-the-loop AI applications," *Human Computation*, vol. 9, no. 1, pp. 1–21, 2022. [Online]. Available: https://doi.org/10.15346/hc.v9i1.134

[12] S. Lambiase, G. Catolino, F. Palomba, F. Ferrucci, and D. Russo, "Investigating the role of cultural values in adopting large language models for software engineering," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 1, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3725529

[13] J. T. Liang, C. Yang, and B. A. Myers, "A large-scale survey on the usability of AI programming assistants: Successes and challenges," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3608128

[14] H. Mozannar, G. Bansal, A. Fourney, and E. Horvitz, "Reading between the lines: Modeling user behavior and costs in AI-assisted programming," in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–16. [Online]. Available: https://doi.org/10.1145/3613904.3641936

[15] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, Apr. 2023. [Online]. Available: https://doi.org/10.1145/3586030

[16] M. C. Davis, E. Aghayi, T. D. Latoza, X. Wang, B. A. Myers, and J. Sunshine, "What's (not) working in programmer user studies?" *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 5, pp. 1–32, 2023. [Online]. Available: https://doi.org/10.1145/3587157

[17] Z. Sharafi, Y. Huang, K. Leach, and W. Weimer, "Toward an objective measure of developers' cognitive activities," *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 3, Mar. 2021. [Online]. Available: https://doi.org/10.1145/3434643

[18] Y. Huang, K. Leach, Z. Sharafi, N. McKay, T. Santander, and W. Weimer, "Biases and differences in code review using medical imaging and eye-tracking: Genders, humans, and machines," in *International Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 456–468. [Online]. Available: https://doi-org.proxy.lib.umich.edu/10.1145/3368089.3409681

[19] I. Bertram, J. Hong, Y. Huang, W. Weimer, and Z. Sharafi, "Trustworthiness perceptions in code review: An eye-tracking study," in *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, ser. ESEM

’20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3382494.3422164

[20] S. Cao and C.-M. Huang, “Understanding user reliance on AI in assisted decision-making,” *Proc. ACM Hum.-Comput. Interact.*, vol. 6, no. CSCW2, Nov. 2022. [Online]. Available: https://doi.org/10.1145/3555572

[21] Z. Wu, Y. Wang, M. Langer, and A. M. Feit, “Releyeance: Gaze-based assessment of users’ AI-reliance at run-time,” 2025. [Online]. Available: https://doi.org/10.1145/3725841

[22] K. Rayner, “Eye movements in reading and information processing,” *Psychological Bulletin*, vol. 85, no. 3, pp. 618–660, 1978. [Online]. Available: https://doi.org/10.1037/0033-2909.85.3.618

[23] Z. Sharafi, T. Shaffer, B. Sharif, and Y.-G. Guéhéneuc, “Eye-tracking metrics in software engineering,” in *2015 Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2015, pp. 96–103. [Online]. Available: https://doi.org/10.1109/APSEC.2015.53

[24] A. Poole and L. J. Ball, “Eye tracking in human-computer interaction and usability research: Current status and future,” in *Encyclopedia of Human-Computer Interaction*, 2005. [Online]. Available: https://cir.nii.ac.jp/crid/1571698600772712320

[25] W. Wang, H. Ning, G. Zhang, L. Liu, and Y. Wang, “Rocks coding, not development: A human-centric, experimental evaluation of llm-supported se tasks,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024. [Online]. Available: https://doi.org/10.1145/3643758

[26] A. Hellas, J. Leinonen, S. Sarsa, C. Koutcheme, L. Kujanpää, and J. Sorva, “Exploring the responses of large language models to beginner programmers’ help requests,” in *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 1*, ser. ICER ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 93–105. [Online]. Available: https://doi.org/10.1145/3568813.3600139

[27] S. MacNeil, A. Tran, A. Hellas, J. Kim, S. Sarsa, P. Denny, S. Bernstein, and J. Leinonen, “Experiences from using code explanations generated by large language models in a web software development e-book,” in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, ser. SIGCSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 931–937. [Online]. Available: https://doi.org/10.1145/3545945.3569785

[28] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, “Using an LLM to help with code understanding,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE ’24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3597503.3639187

[29] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS ’23. Red Hook, NY, USA: Curran Associates Inc., 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html

[30] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, “A systematic evaluation of large language models of code,” in *Proceedings of the 6th ACM SIGPLAN international symposium on machine programming*, 2022, pp. 1–10. [Online]. Available: https://doi.org/10.1145/3520312.3534862

[31] J. Prather, B. N. Reeves, P. Denny, B. A. Becker, J. Leinonen, A. Luxton-Reilly, G. Powell, J. Finnie-Ansley, and E. A. Santos, ““it’s weird that it knows what i want”: Usability and interactions with copilot for novice programmers,” *ACM Trans. Comput.-Hum. Interact.*, vol. 31, no. 1, Nov. 2023. [Online]. Available: https://doi.org/10.1145/3617367

[32] P. Vaithilingam, T. Zhang, and E. L. Glassman, “Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models,” in *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*, ser. CHI EA ’22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: https://doi.org/10.1145/3491101.3519665

[33] A. Shah, A. Chernova, E. Tomson, L. Porter, W. G. Griswold, and A. G. Soosai Raj, “Students’ use of github copilot for working with large code bases,” in *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1*, ser. SIGCSETS 2025. New York, NY, USA: Association for Computing Machinery, 2025, p. 1050–1056. [Online]. Available: https://doi.org/10.1145/3641554.3701800

[34] Q. Ma, T. Wu, and K. Koedinger, “Is AI the better programming partner? human-human pair programming vs. human-AI pair programming,” *arXiv preprint arXiv:2306.05153*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2306.05153

[35] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do users write more insecure code with AI assistants?” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 2785–2799. [Online]. Available: https://doi.org/10.1145/3576915.3623157

[36] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, “Lost at c: a user study on the security implications of large language model code assistants,” in *Proceedings of the 32nd USENIX Conference on Security Symposium*, ser. SEC ’23. USA: USENIX Association, 2023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval

[37] S. Passi and M. Vorvoreanu, “Overreliance on AI literature review,” *Microsoft Research*, vol. 339, p. 340, 2022. [Online]. Available: https://www.microsoft.com/en-us/research/publication/overreliance-on-ai-literature-review/

[38] M. A. Just and P. A. Carpenter, “A theory of reading: from eye fixations to comprehension.” *Psychological review*, vol. 87, no. 4, p. 329, 1980. [Online]. Available: https://doi.org/10.1037/0033-295X.87.4.329

[39] C. Tona, R. Juárez-Ramírez, S. Jiménez, and M. Durán, “Exploring llm tools through the eyes of industry experts and novice programmers,” in *2024 12th International Conference in Software Engineering Research and Innovation (CONISOFT)*, 2024, pp. 313–321. [Online]. Available: https://doi.org/10.1109/CONISOFT63288.2024.00048

[40] GitHub. (2024) About github. Accessed: 2025-05-14. [Online]. Available: https://github.com/newsroom/press-releases/github-universe-2024

[41] J. H. Goldberg and J. I. Helfman, “Comparing information graphics: A critical look at eye tracking,” in *BEyond Time and Errors: Novel evaLuation Methods for Information Visualization*, 2010. [Online]. Available: http://doi.acm.org/10.1145/2110192.2110203

[42] Z. Sharafi, B. Sharif, Y.-G. Guéhéneuc, A. Begel, R. Bednarik, and M. Crosby, “A practical guide on conducting eye tracking studies in software engineering,” *Empirical Software Engineering*, pp. 1–47, 2020. [Online]. Available: https://doi.org/10.1007/s10664-020-09829-4

[43] J. W. Hardin and J. M. Hilbe, *Generalized estimating equations*. chapman and hall/CRC, 2002.

[44] J. O. Wobbrock, L. Findlater, D. Gergle, and J. J. Higgins, “The aligned rank transform for nonparametric factorial analyses using only ANOVA procedures,” in *Human factors in computing systems*, 2011. [Online]. Available: https://doi.org/10.1145/1978942.1978963

[45] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The impact of AI on developer productivity: Evidence from github copilot,” *arXiv preprint arXiv:2302.06590*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2302.06590

[46] S. J. Spencer, C. M. Steele, and D. M. Quinn, “Stereotype threat and women’s math performance,” *Journal of experimental social psychology*, vol. 35, no. 1, pp. 4–28, 1999. [Online]. Available: https://doi.org/10.1006/jesp.1998.1373

[47] J. R. Shapiro and S. L. Neuberg, “From stereotype threat to stereotype threats: Implications of a multi-threat framework for causes, moderators, mediators, consequences, and interventions,” *Personality and Social Psychology Review*, vol. 11, no. 2, pp. 107–130, 2007. [Online]. Available: https://doi.org/10.1177/1088868306294790

[48] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. E. Emam, and J. Rosenberg, “Preliminary guidelines for empirical research in software engineering,” *IEEE Transactions on Software Engineering*, vol. 28, no. 8, pp. 721–734, Aug. 2002. [Online]. Available: https://doi.org/10.1109/TSE.2002.1027796

[49] G. M. Sullivan and R. Feinn, “Using effect size—or why the p value is not enough,” *Journal of graduate medical education*, vol. 4, no. 3, pp. 279–282, 2012. [Online]. Available: https://doi.org/10.4300/JGME-D-12-00156.1